

Gradient Descent as a Teaching and Learning Model

Bruce DeSimas

May 24, 2026

Overview

This document summarizes the use of a mental model for teaching by using gradient descent as the model. The emphasis is on high-level steps including the mean squared error (MSE) loss function, partial derivatives, caching in backpropagation, learning rate, batch size, and how numerical gradient checking fits in as an independent validation method.

A Note on the Central Conjecture

The deeper aim of this paper is to draw out the parallels between traditional teaching, self-directed learning, accelerated learning, motivation, high standards, and mastery on one hand, and the mechanics of machine learning on the other. The central conjecture is this: the best way to begin is to randomly initialize the model — one's understanding — and jump in. Point the learning process at a pile of data: books, problems, anything available. Just as in stochastic gradient descent, when a reasonable set of hyperparameters is chosen, large early error terms are not a setback; they are the signal that forces the learning process onto the right path, provided motivation and stress are managed well.

This reframing — that high error early on is good and necessary — has the potential to change how we think and feel about learning generally. Infants and young children are, in a real sense, remarkably efficient learners, acquiring language, motor control, and a working model of the physical world in a few short years through nothing but trial, error, and correction — with no formal instruction at all. Whatever is happening in that process, it is operating on a completely different principle than a classroom. The natural question is why so much of that capacity seems to fade by adulthood. Part of the answer may lie in how learning is structured later in life: institutional education does not resemble an optimized machine-learning training process, and may in fact work against the very mechanism — error, correction, repeat — that made early learning so effective in the first place.

If there is one lesson to take from machine learning, it is this: start, let errors guide the discovery, embrace them, require them, reward them — and then drive to reduce them, not to avoid them in the first place. Applying this parallel in schools would require rethinking the reward and punishment structures that currently govern how mistakes are treated. This paper introduces that idea. It is not entirely new — elements of it appear in other methods — but rarely to this extent, and rarely grounded explicitly in the mechanics of how modern machine learning models are trained.

This paper is, in short, advocating for the deliberate use of failure to accelerate learning and strengthen memory. Consider a simple test: can you recall the thirteenth problem from the second exam of your second-semester physics course? Almost certainly not — and if you never took such

a course, the example still works with anything from your own past. Now go back ten, twenty, or thirty years and recall a real failure: a disaster on a test, a serious mistake in a job, the shock and discomfort of it, and then the process of correcting it. That memory is likely still vivid, and the correction process taught you something no lecture could have. This is the effect that “learn by doing” approaches strive for, and partially achieve.

A particularly interesting consequence of this reframing concerns the use of AI in education. If honest mistakes were the rewarded unit of work — not the polished final answer — the incentive to have an AI produce a perfect report would largely disappear, because a perfect report would earn no credit. Credit would instead flow from the mistake itself and, more importantly, from the subsequent correction: the update to the model. In machine-learning terms, a perfect prediction with zero error produces zero gradient — there is nothing to learn from. A submission with no errors is, in this framing, a submission with no learning in it. Restructuring assessment around this idea — rewarding the error-and-correction cycle rather than the error-free output — is a direction this paper believes deserves serious exploration, and one where the parallel to machine learning is most direct.

None of this requires that learning be dramatic or painful. The point is narrower: making mistakes is not a failure of the learning process, it *is* the learning process. If honest mistakes were rewarded as the raw material of learning, the incentive structure around cheating would shift fundamentally. Asking an AI to produce a perfect report would earn zero credit, because credit would flow from the mistake and, more importantly, from the subsequent correction — the update to the model. This is how learning works in machine learning, and this paper proposes it as a new mental model for teaching and learning generally.

The Generalized Teaching Challenge

Zooming out and taking a fresh look at teaching, it becomes obvious that modern day machine learning provides some useful analogies for a technologist, mathematician, chemist, or physicist to go from practitioner to mentor in a compressed timeframe. The text *The Seven Laws of Teaching*, John Gregory, 1886, provides a clear roadmap to teaching. In the last 140 years, a significant volume of textbooks and publications have been printed, yet nothing changes the essence of what this classic teaches us. It is therefore the aim of this paper to focus on the essentials: the student, the teacher, and the goal of learning. This is the entire system, nothing less, nothing more. We have several components such as learning rate, making mistakes, neuroplasticity, pacing, iterations, scope, progression, rhythm, and making adjustments by nudging, steering, and guiding the student toward mastery. From this, it can be shown that the same ingredients can be found in machine learning. This paper aims to reveal the use of machine learning as a powerful analogy tool for the new or experienced teacher.

The Foundation of Machine Learning

For a simple regression model with two input features and one target, the prediction is formed by a weighted sum plus a bias:

$$\hat{y} = w_1x_1 + w_2x_2 + b \tag{1}$$

The weights are commonly initialized to small random values, while the bias can start at 0.0.

The mean squared error measures the average squared difference between predictions and targets:

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 \quad (2)$$

MSE is always nonnegative, and its minimum is zero only when every prediction matches its target exactly.

High-Level Training Loop

The standard gradient-descent loop for this model is:

- Initialize weights to small random numbers and bias to 0.0.
- Run the forward pass to compute predictions for the current batch.
- Compute residuals and then MSE over the batch.
- Compute partial derivatives of the loss with respect to each weight and the bias.
- Update parameters by moving opposite the gradient using the learning rate α .
- Repeat until a stopping rule is met: small gradient, little improvement in loss, or maximum iteration count.

Batch size determines how many examples are used to estimate the loss and gradient before each update. Smaller batches tend to be noisier but more responsive; larger batches give smoother, more stable updates.

Partial Derivatives

The critical derivatives for the MSE loss in linear regression are:

$$\frac{\partial \text{MSE}}{\partial w_j} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij} \quad (3)$$

$$\frac{\partial \text{MSE}}{\partial b} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \quad (4)$$

A larger error produces a larger gradient signal, which is why early large errors are not inherently bad — they often provide strong directional information for correction.

Forward Pass, Cache, and Backpropagation

In a simple model, the forward pass is conceptually straightforward: compute predictions, residuals, and loss. The backward pass applies the chain rule to compute how the loss changes with respect to each parameter.

Caching means saving intermediate values from the forward pass — predictions, residuals, activations, pre-activations — so they can be reused efficiently during backpropagation. In deep networks, this cache exists layer by layer because each layer’s outputs are needed to compute gradients during the reverse pass.

The Caching Analogy — Expanded

The caching mechanism in backpropagation is richer than it first appears, and its teaching analogy operates at three distinct layers — each corresponding to a different actor in the learning system.

Layer 1 — The Teacher’s Cache

During instruction, an effective teacher continuously observes and retains intermediate signals: a question asked at the wrong moment, a hesitation before answering, a confident but incorrect response, a look of recognition when a concept lands. These are cached signals from the forward pass of the lesson. The backward pass — the teacher’s adjustment of pacing, vocabulary, example choice, and depth — depends entirely on what was captured going forward. A teacher who does not cache, who lectures without observing, has no gradient signal to work with. The backward pass is blind.

Layer 2 — The Student’s Cache

The student’s working memory during a lesson is a cache of intermediate representations: partial understanding, tentative connections between new and prior knowledge, unresolved questions held in suspension. If cognitive load exceeds the capacity of this cache, the forward pass completes but the backward pass — consolidation, practice, sleep-dependent memory formation — has nothing useful to work with. Cognitive Load Theory describes exactly this mechanism: not that the lesson was too hard in absolute terms, but that the cache overflowed before intermediate representations could be formed. Chunking is not merely a convenience — it is cache management.

Layer 3 — Formative Assessment as Explicit Cache Writes

Exit tickets, think-pair-share, quick checks for understanding, and cold-calling are deliberate cache write operations. The teacher forces the student’s intermediate state to be externalized and saved before the lesson advances. Without these explicit writes, the session may appear to complete successfully while the student’s internal cache was never fully populated. Formative assessment makes the invisible cache visible — to both teacher and student — so that the backward pass of learning has accurate, timely gradient information to work with.

A concise restatement: *formative assessment is the teacher explicitly writing to the cache so that the backward pass of learning — consolidation, practice, correction — has something to work with.*

Matrix Operations and Implementation

Much of the implementation complexity in deep learning comes from bookkeeping: tracking tensor shapes, cached intermediates, and the order of gradient computations. The numerical work is usually dominated by matrix operations that compactly express what would otherwise be nested loops of multiply-and-sum operations.

Numerical Gradient Checking

Numerical gradient checking estimates a derivative by nudging a parameter slightly upward and downward and measuring the resulting change in the loss:

$$\frac{\partial J}{\partial \theta} \approx \frac{J(\theta + \epsilon) - J(\theta - \epsilon)}{2\epsilon} \quad (5)$$

This method is too slow for training but valuable as an independent check on analytical gradients. In the teaching analogy, it corresponds to the teacher asking a slightly harder and slightly easier version of the same question — measuring how the student’s response changes — to independently verify the direction and magnitude of understanding. It is Socratic probing used as a validation tool.

Teaching Analogy

The following list maps optimization ideas to teaching and learning:

- **Error / loss:** The gap between current understanding and mastery.
- **Gradient:** The direction and strength of the next instructional adjustment.
- **Parameters:** The parts of understanding that are being shaped or tuned.
- **Hyperparameters:** Teaching choices set ahead of time — pacing, chunk size, learning rate.
- **Learning rate:** How large each instructional nudge should be.
- **Batch size / chunk size:** How much material is processed before adjusting instruction.
- **Large early error:** A strong feedback signal that accelerates correction at a reasonable learning rate.
- **Small steady updates:** Gradual improvement that avoids overshooting understanding.
- **Large chunk:** More context and connections, but greater risk of cognitive overload.
- **Small chunk:** Less overload and clearer correction, sometimes with less big-picture coherence.
- **Cache / formative assessment:** Intermediate states saved during the forward pass so the backward pass has gradient information. In teaching: exit tickets, quick checks, think-pair-share — deliberate writes to the student’s working cache before the lesson advances.
- **Assumptions and constraints in science:** Simplifications that make a problem tractable — like reducing chunk size to isolate a learnable part of a larger system.

Conclusion

Good teaching, like stochastic gradient descent, uses feedback from imperfect understanding to steer toward mastery without changing too abruptly. Making errors is essential — as with the MSE function, the error curve is the key to learning; without errors, no learning occurs. The caching mechanism — spanning teacher observation, student working memory, and formative assessment — is perhaps the richest single point of correspondence between the two domains. It suggests

that the bottleneck in learning is rarely the quality of instruction in isolation, but the fidelity with which intermediate states are captured, preserved, and made available for the backward pass of consolidation and correction.

Cognitive Load Theory, Educational Research, and Instructional Design: Some Food for Thought, de Jong, Ton, *Instructional Science*, v38 n2 p105-134, Mar 2010.

Gradient Descent, Google Machine Learning Crash Course. <https://developers.google.com/machine-learning/crash-course/linear-regression/gradient-descent>

Krishnamurthy, Krishna. *Math and Architectures of Deep Learning*. Manning Publications, 2022.

Stevens, Eli, Antiga, Luca, and Viehmann, Thomas. *Deep Learning with PyTorch*. Manning Publications, 2020.